

Brain Tumor MRI Segmentation

Baseline vs. CNN vs. U-Net, with Model Interpretability and MLOps

Ahmed Mohamed Aly Mikaela Louise Barila Lavilla Sushree Sarita Pradhan

Machine Learning for Data Analytics — Group Project

14 June 2026

Abstract

We address binary segmentation of brain tumors in MRI slices. Three approaches of increasing capacity are compared: a non-learning intensity *baseline*, a small *CNN*, and a *U-Net*. The full workflow is implemented as a reproducible, notebook-driven pipeline with experiment tracking in MLflow. Beyond the final numbers, we document the *experimental journey* in detail, including a training collapse of the U-Net caused by class imbalance and how it was diagnosed and fixed. After switching the U-Net to a pure Dice objective, it reaches a validation Dice of **0.842** (IoU 0.733), substantially outperforming the CNN (Dice 0.173) and the baseline (Dice 0.068). Finally, SHAP and LIME are used to explain the model's predictions. All runs were executed on a rented GPU and are fully reproducible.

Contents

1	Introduction	1
2	Dataset and Exploratory Analysis	1
3	Preprocessing and Feature Engineering	2
4	Methods	2
4.1	Metrics	2
4.2	Models	2
5	Experimental Setup	2
5.1	Protocol	2
5.2	Tooling, tracking, and hardware	3
6	Experimental Journey	3
6.1	Iteration 1 — Under-trained smoke test	3
6.2	Iteration 2 — Environment mismatch (MLflow file store)	3
6.3	Iteration 3 — U-Net training collapse	3
6.4	Iteration 4 — Fix and final run	3
7	Results	4
8	Model Interpretability	4
8.1	SHAP — gradient-based pixel attribution	4
8.2	LIME — superpixel perturbation	5
8.3	SHAP vs. LIME	6
9	MLOps and Reproducibility	7

10 Lessons Learned	8
11 Conclusion and Future Work	8
A Final Run Configuration	8

1 Introduction

Accurate delineation of tumor regions in MRI is a clinically important and technically challenging segmentation problem: tumors occupy only a small fraction of each image, producing severe foreground/background class imbalance. The goal of this project is to build and compare several segmentation approaches on a brain-tumor MRI dataset, to track all experiments reproducibly (MLOps), and to interpret *why* the best model makes its predictions (SHAP and LIME). This report records the data, methods, the experimental process (including failures and fixes), the final results, and the lessons learned.

2 Dataset and Exploratory Analysis

The dataset consists of **3064** MRI slices, each paired with a binary tumor segmentation mask. Exploratory analysis established the following:

- All slices contain a tumor region; the tumor is a *small fraction* of the total pixels, i.e. the task is strongly class-imbalanced.
- Pixel-intensity distributions motivate normalization to a common range.
- Tumor-size distribution varies widely across slices, which interacts with the imbalance and motivates the use of overlap-based metrics (Dice, IoU) rather than pixel accuracy.

3 Preprocessing and Feature Engineering

Each raw image and mask is processed by `src/preprocessing.ipynb` as follows:

1. **Resize** all images and masks to 256×256 (nearest-neighbour for masks to preserve binary labels).
2. **Normalize** image pixel values to $[0, 1]$ ($x/255$).
3. **Binarize** masks to $\{0, 1\}$ (tumor vs. background).
4. Save as `.npy` arrays under `data/processed/`.

The processed tensors are consumed by a custom `BrainTumorDataset` that returns image tensors of shape $[3, 256, 256]$ and masks of shape $[1, 256, 256]$.

4 Methods

4.1 Metrics

For a binary prediction P (thresholded at 0.5) and ground-truth mask G , with smoothing constant ϵ :

$$\text{Dice}(P, G) = \frac{2|P \cap G| + \epsilon}{|P| + |G| + \epsilon}, \quad \text{IoU}(P, G) = \frac{|P \cap G| + \epsilon}{|P \cup G| + \epsilon}. \quad (1)$$

The training objective is the soft Dice loss computed on probabilities p :

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i p_i g_i + \epsilon}{\sum_i p_i + \sum_i g_i + \epsilon}. \quad (2)$$

4.2 Models

Baseline (threshold). A non-learning reference that converts the image to grayscale and thresholds intensity at 0.5. It establishes the floor a learned model must beat.

CNN (SimpleCNN). A small fully-convolutional network: $\text{Conv}(3 \rightarrow 16) \rightarrow \text{ReLU} \rightarrow \text{Conv}(16 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{Conv}(32 \rightarrow 1) \rightarrow \sigma$, with 3×3 kernels. It can learn local texture but lacks multi-scale context for precise localization.

U-Net. An encoder-decoder with skip connections. Encoder feature widths are [64, 128, 256, 512] with a 1024-channel bottleneck; each stage is a *DoubleConv* block (Conv-BatchNorm-ReLU, twice). The decoder upsamples with transposed convolutions and concatenates the corresponding encoder feature maps (skip connections), enabling precise, multi-scale localization. A final 1×1 convolution followed by a sigmoid yields the per-pixel tumor probability.

5 Experimental Setup

5.1 Protocol

The dataset is split 80/20 into **2451** training and **613** validation samples using a *fixed seed* (42) for reproducibility. Training uses the Adam optimizer, a `ReduceLROnPlateau` scheduler (on validation Dice), early stopping, and *best-checkpoint* selection (the weights with the highest validation Dice are saved and restored). Validation Dice and IoU are logged every epoch.

5.2 Tooling, tracking, and hardware

All experiments are tracked in **MLflow** (parameters, per-epoch metrics, and model artifacts). The pipeline is fully path-portable (it locates the project root automatically) and runs unchanged on Windows or Linux. Final runs were executed on a rented **NVIDIA RTX PRO 6000 (Blackwell)** GPU via `vast.ai`, using PyTorch 2.12 / CUDA 13. To minimize transfer, only the raw images (~333 MB) were uploaded and preprocessing was re-run on the GPU host to regenerate the processed arrays.

6 Experimental Journey

This section documents the iterations, including failures, because they are central to the learning outcomes.

6.1 Iteration 1 — Under-trained smoke test

Initial runs used only **2 epochs on CPU**, yielding Dice of 0.06/0.13/0.21 for baseline/CNN/U-Net. These numbers reflect *under-training*, not model capacity; they motivated a proper training protocol (more epochs, early stopping, GPU).

6.2 Iteration 2 — Environment mismatch (MLflow file store)

On the GPU host, a newer MLflow version *hard-errors* on the file-store backend that older versions only warned about. The fix was to explicitly opt in (`MLFLOW_ALLOW_FILE_STORE=true`) so the same notebook runs across MLflow versions. *Lesson: pin behaviour you depend on; environments drift.*

6.3 Iteration 3 — U-Net training collapse

With a combined **BCE+Dice** loss, the U-Net *collapsed*: it learned to predict “background everywhere,” which minimizes the BCE term under heavy class imbalance while making the segmentation useless. The signature is a *falling loss with a vanishing task metric*:

Epoch	Val Dice	Train loss
1–6	0.02 → 0.07	decreasing (learning)
7	≈ 0.0001	still decreasing
8–18	≈ 0.0001	still decreasing

The CNN, which used a *pure Dice* loss, did not collapse — a key diagnostic clue.

6.4 Iteration 4 — Fix and final run

The U-Net loss was switched to **pure Dice** (predicting all-background yields a *bad* Dice score, so the optimizer is repelled from the degenerate solution), mixed precision was disabled to remove a variable, and the epoch cap was raised to 60 (patience 15). The U-Net then trained cleanly:

Epoch	1	6	30	Best epoch (58)	Val Dice
Val Dice	0.480	0.727	0.831	early-stopped	0.8415

7 Results

Table 1 reports final validation performance (best checkpoint).

Table 1: Final validation results (best checkpoint, seed 42, 613 validation slices).

Model	Dice	IoU	Epochs run
Baseline (threshold)	0.068	0.036	—
CNN (SimpleCNN)	0.173	0.098	40 (early-stopped)
U-Net	0.842	0.733	58 (early-stopped)

The ranking is the expected one: **U-Net** $\gg$ **CNN** $>$ **Baseline**. The U-Net’s encoder–decoder structure with skip connections captures both global context and fine spatial detail, which the shallow CNN cannot, and which a non-learning threshold fundamentally cannot.

8 Model Interpretability

We explain the trained U-Net with two complementary methods.

8.1 SHAP — gradient-based pixel attribution

To keep attributions tumor-focused, the model output is reduced to the *mean predicted probability inside the predicted tumor region* (not the whole-image mean, which is dominated by background and dilutes the signal). `shap.GradientExplainer` attributes this score to input pixels relative to a 20-image background set, evaluated on the largest-tumor validation slices; the maps are Gaussian-smoothed for display. Positive (red) pixels increase the predicted tumor score, negative (blue) suppress it, and the green contour marks the predicted tumor (Figure 1).

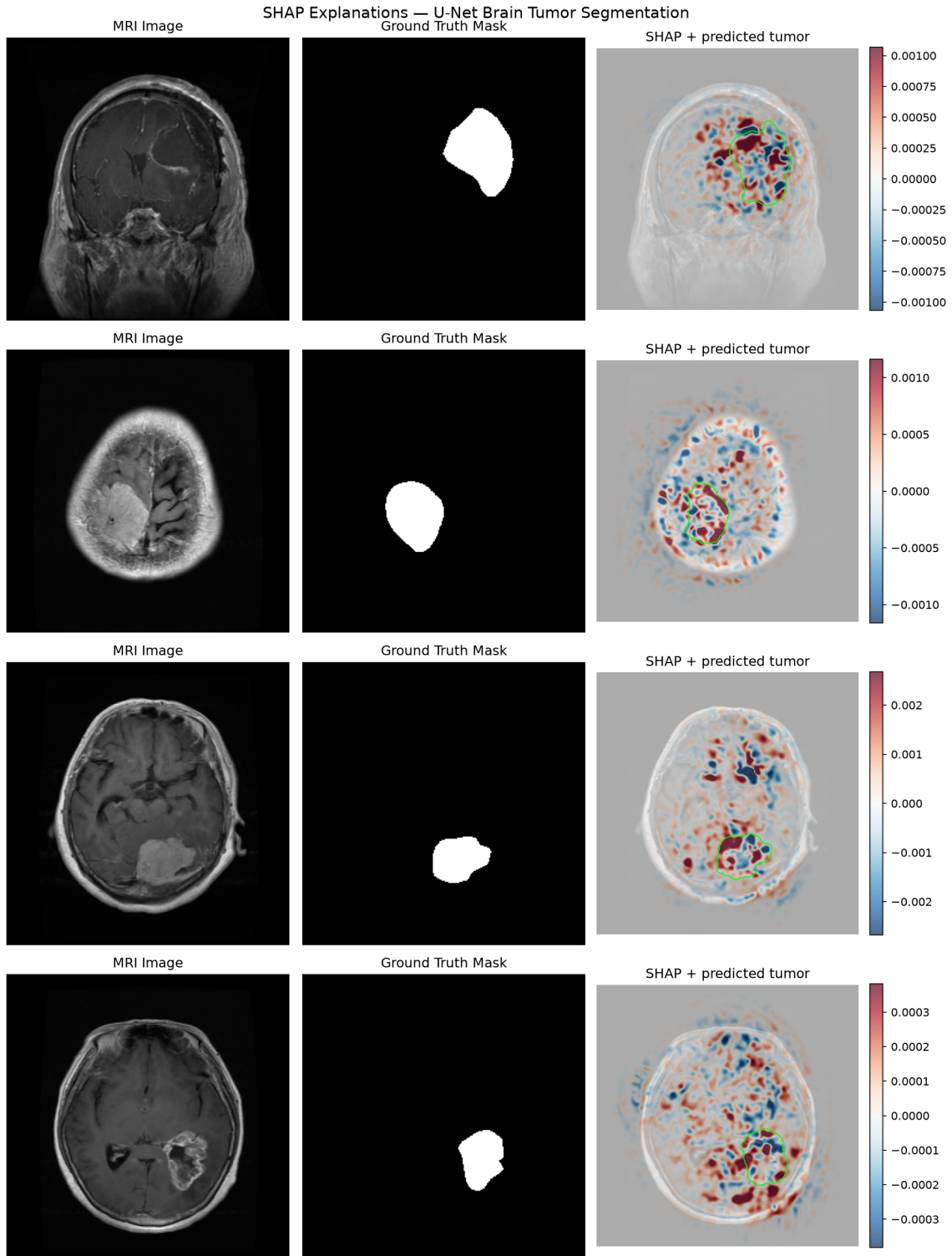


Figure 1: SHAP attributions (smoothed) for the U-Net on the largest-tumor validation cases. Attribution mass falls on brain tissue in and around the predicted tumor (green contour); the pixel-level signal is fine-grained and somewhat noisy, as is typical of gradient-based attribution.

8.2 LIME — superpixel perturbation

`LimeImageExplainer` perturbs superpixels and measures the change in the same tumor-region score (1000 samples per image). The superpixels the model relies on most are highlighted (Figure 2).

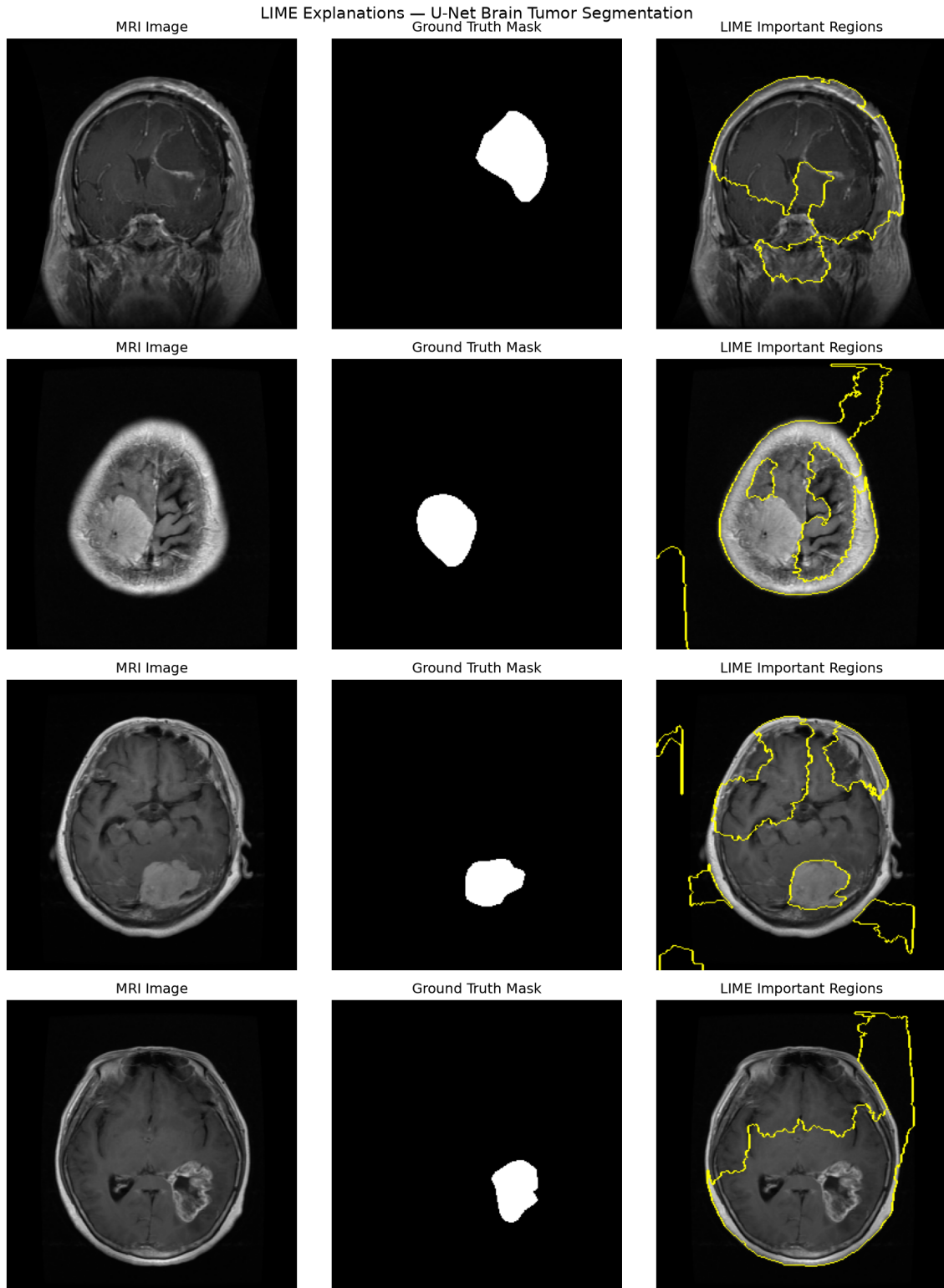


Figure 2: LIME important superpixels. The highlighted regions overlap the tumor area, giving a coarser but more human-readable, region-level explanation than SHAP.

8.3 SHAP vs. LIME

Figure 3 shows both for the same slices. SHAP gives fine pixel-level detail (necessarily noisier), while LIME gives coarse region-level evidence; both localize to the tumor area, and their agreement there increases confidence that the model attends to tumor-relevant evidence rather than

artifacts.

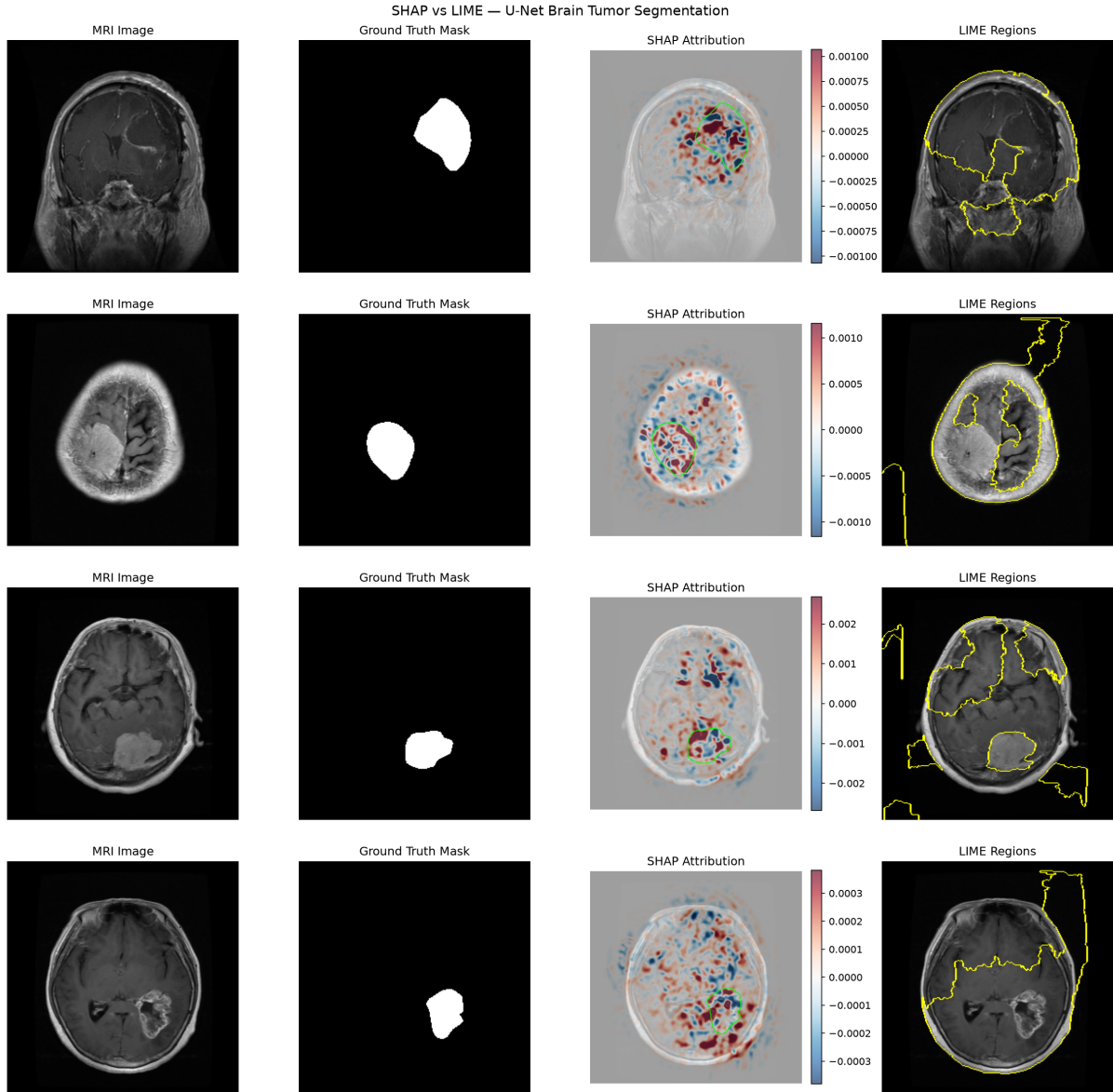


Figure 3: Side-by-side SHAP and LIME explanations for the U-Net.

9 MLOps and Reproducibility

- **Experiment tracking:** every run (baseline, CNN, U-Net, and the failed iterations) is logged to MLflow with parameters, per-epoch metrics, and artifacts.
- **Reproducibility:** fixed random seed for the train/validation split; deterministic, version-robust MLflow configuration; portable path resolution so the notebooks run from any working directory on Windows or Linux.
- **Checkpointing:** the best model by validation Dice is saved and restored, decoupling the reported result from the last (possibly worse) epoch.

10 Lessons Learned

1. **Under-training masks capability.** Two-epoch CPU runs made good models look broken; never judge a model from a smoke test.
2. **Track the task metric, not just loss.** A falling loss hid the U-Net collapse; per-epoch validation Dice exposed it.
3. **Class imbalance breaks BCE.** For tiny targets, a Dice (or Tversky/Focal) objective is far more robust than binary cross-entropy.
4. **Best-checkpoint + early stopping** turn a failure into a diagnosable, recoverable event instead of a silently bad final model.
5. **Environment drift is real.** The same library changed from warning to hard error across versions; pin the behaviour you rely on.
6. **Move the small artifact, compute the big one on the fast machine.**

11 Conclusion and Future Work

A U-Net trained with a Dice objective segments brain tumors with a validation Dice of **0.842**, decisively outperforming a simple CNN and a non-learning baseline, and its predictions are explained consistently by both SHAP and LIME. The project is fully tracked and reproducible. Future work: data augmentation (flips/rotations) for further generalization, an imbalance-aware loss (Tversky/Focal) comparison, a held-out test split in addition to validation, and a larger hyperparameter sweep.

A Final Run Configuration

```
Split:          2451 train / 613 val (seed 42, 80/20)
Batch size:     8
Optimizer:      Adam    (CNN lr 1e-3, U-Net lr 1e-4)
Scheduler:      ReduceLROnPlateau (mode=max on val Dice)
Loss:           Dice    (both CNN and U-Net)
Epoch cap:     60      Early stopping patience: 15
Mixed prec.:    off
Hardware:       NVIDIA RTX PRO 6000 (Blackwell), PyTorch 2.12 / CUDA 13
Tracking:       MLflow (mlruns/)
Result:         Baseline 0.068 | CNN 0.173 | U-Net 0.842 (Dice)
```